



UNIVERSIDAD
TORCUATO DI TELLA

Licenciatura en Tecnología Digital

Tecnología Digital VI: Inteligencia Artificial

Programación de backpropagation

Clase práctica 17

Motivación

- ¿Cómo se representan las operaciones de backpropagation de forma **matricial**?
- ¿Cómo se programa esta estrategia de backpropagation con **NumPy**?
- ¿Cómo se implementa para un **conjunto de datos específico** de forma general para **N capas y M neuronas**?

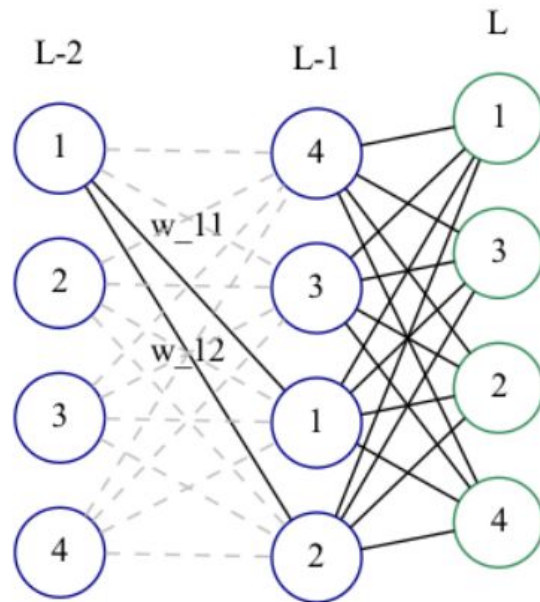
Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'
Ecuaciones generales	08:05	09:55	20'
Ejercicio red fija	08:25	10:15	40'
Ejercicio red genérica	09:05	10:55	30'
Cierre	09:35	11:25	05'
Fin de la clase	09:40	11:30	-

Ecuaciones generales | Repaso de la estrategia

Dados los X , Y correspondientes al *input* y la salida esperada en entrenamiento:

- 1) Hacer pasada **forward**, partiendo de X .
- 2) Calcular las **derivadas** respecto a los pesos y los sesgos, partiendo del error entre Y e Y_{pred} .
- 3) **Actualizar** pesos y sesgos.



Ecuaciones generales | Repaso de la notación

$w_{i,j}^l$ →

Peso entre la neurona ***i*** de la capa ***l* - 1** y la neurona ***j*** de la capa ***l***.

Entrada z de una neurona j para la capa l :

$$z_j^l = \sum_i w_{i,j} \cdot a_i^{l-1} + b_j^l$$

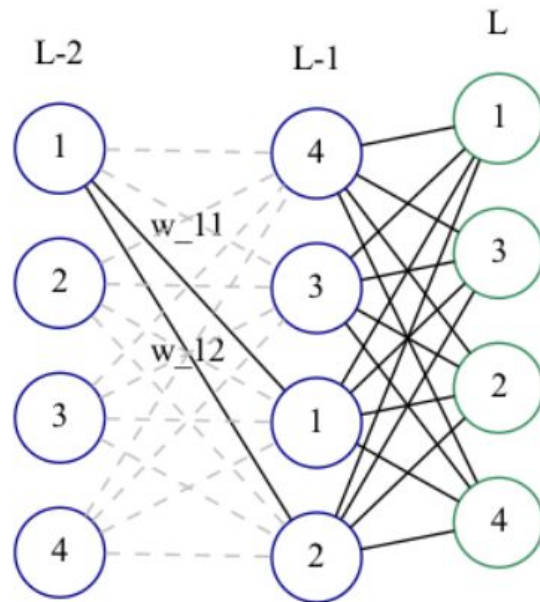
→ **Entrada neta** de una neurona.

Donde $a_i^l = \sigma(z_i^l)$ →

Salida de una neurona (entrada neta pasada por la función de activación).

b_j^l →

Sesgo de la neurona ***j*** capa ***l***.



Ecuaciones generales | Las fórmulas

$$\frac{\partial L}{\partial w_{ij}} = \delta_j^l \cdot a_i^{l-1} \longrightarrow$$

Derivada del **costo** respecto
a un **peso**.

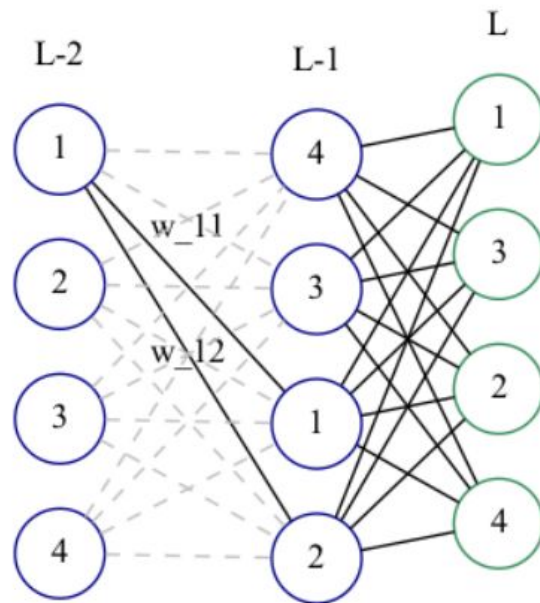
$$\frac{\partial L}{\partial b_j} = \delta_j^l \longrightarrow$$

Derivada del **costo** respecto
a un **sesgo**.

$$\begin{aligned} \delta_j^L &= \frac{\partial C}{\partial z_j^L} \\ &= (a_j^L - y_j^L) \cdot \sigma'(z_j^L) \end{aligned}$$

$$\begin{aligned} \delta_j^l &= \frac{\partial C}{\partial z_j^l} \\ &= \sigma'(z_j^l) \sum_k \delta_k^{l+1} w_{jk}^{l+1} \end{aligned}$$

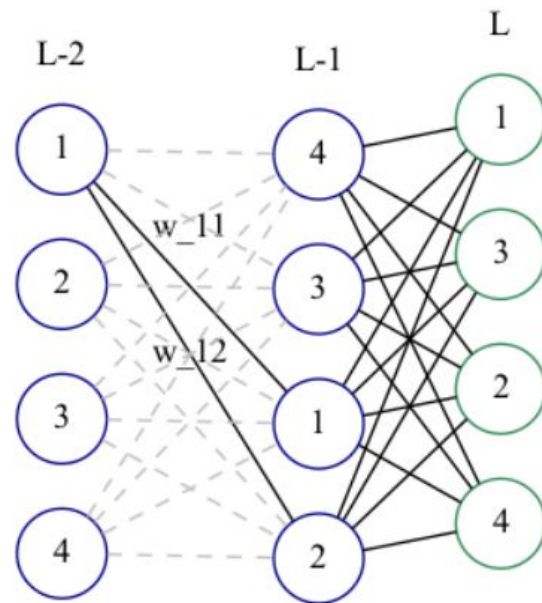
Derivada del
costo respecto
al **input** de una
neurona.



Ecuaciones generales | Las fórmulas (cont.)

$$w_{i,j}^l = w_{i,j}^l - \mu \frac{\partial L}{\partial w_{i,j}^l} \longrightarrow \text{Actualización de los pesos.}$$

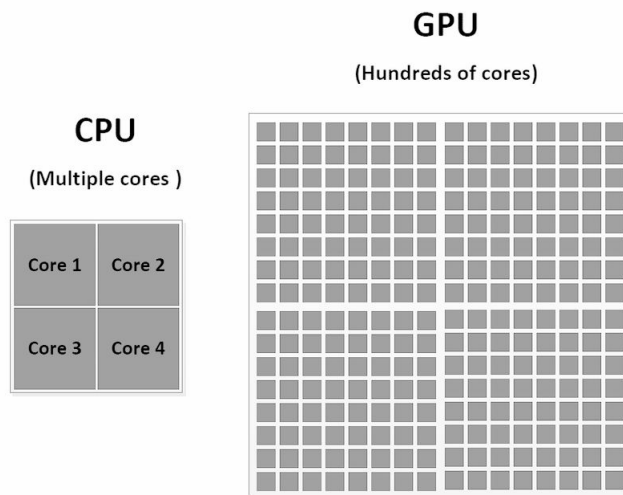
$$b_j^l = b_j^l - \mu \frac{\partial L}{\partial b_j^l} \longrightarrow \text{Actualización de los sesgos.}$$



Ecuaciones generales | ¿Por qué con matrices?

Backpropagation involucra realizar reiterados cálculos que son **costosos** computacionalmente.

Las **GPU** son capaces de realizar cálculos de matrices con un mayor nivel de **paralelización** que las CPU. Por lo tanto, tal representación resulta ideal para entrenar múltiples lotes de datos.



Ecuaciones generales | Representación matricial

Queremos procesar múltiples observaciones de entrenamiento a la vez. Por lo tanto, tenemos que cambiar un poco la implementación...

$$X = \begin{bmatrix} x_a^1 & x_b^1 & \dots & x_m^1 \\ x_a^2 & x_b^2 & \dots & x_m^2 \\ \dots & \dots & \dots & \dots \\ x_a^n & x_b^n & \dots & x_m^n \end{bmatrix}$$

Observaciones de
entrenamiento
representadas como matriz.

$$W^{l+1} = \begin{bmatrix} w_{11}^{l+1} & \dots & w_{1n}^{l+1} \\ \vdots & \ddots & \vdots \\ w_{m1}^{l+1} & \dots & w_{mn}^{l+1} \end{bmatrix}$$

Matriz de **pesos** entre
la capa **l** y **$l+1$** .

$$B^l = \begin{bmatrix} b_1^l \\ b_2^l \\ \dots \\ b_n^l \end{bmatrix}$$

Vector de **sesgos**
para la capa **l** .

Ecuaciones generales | Repr. matricial (cont.)

Forward propagation para la capa inicial.

$$z^l = X.W^l + B^l$$

NumPy hace
broadcast.

$$= \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,m} \\ x_{2,1} & x_{2,2} & \dots & x_{2,m} \\ \dots & & & \\ x_{n,1} & x_{n,2} & \dots & x_{n,m} \end{bmatrix} \cdot \begin{bmatrix} w_{1,1}^l & w_{1,2}^l & \dots & w_{1,k}^l \\ w_{2,1}^l & w_{2,2}^l & \dots & w_{2,k}^l \\ \dots & & & \\ w_{m,1}^l & w_{m,2}^l & \dots & w_{m,k}^l \end{bmatrix} + \begin{bmatrix} b_1^l \\ b_2^l \\ \dots \\ b_k^l \end{bmatrix}$$

¿Qué cambia para las siguientes capas?

Ecuaciones generales | Repr. matricial (cont.)

Cada fila es una observación de entrenamiento distinta.

Representación de
las **entradas netas**
para la capa l .

$$z^l = \begin{bmatrix} z_{1,1}^l & z_{1,2}^l & \dots & z_{1,m}^l \\ z_{2,1}^l & z_{2,2}^l & \dots & z_{2,m}^l \\ \dots & & & \\ z_{n,1}^l & z_{n,2}^l & \dots & z_{n,m}^l \end{bmatrix}$$

Representación de
los **deltas** para la
capa l .

$$\delta^l = \begin{bmatrix} \delta_{1,1}^l & \delta_{1,2}^l & \dots & \delta_{1,m}^l \\ \delta_{2,1}^l & \delta_{2,2}^l & \dots & \delta_{2,m}^l \\ \dots & & & \\ \delta_{n,1}^l & \delta_{n,2}^l & \dots & \delta_{n,m}^l \end{bmatrix}$$

Representación de
las **salidas** para la
capa l .

$$a^l = \begin{bmatrix} a_{1,1}^l & a_{1,2}^l & \dots & a_{1,m}^l \\ a_{2,1}^l & a_{2,2}^l & \dots & a_{2,m}^l \\ \dots & & & \\ a_{n,1}^l & a_{n,2}^l & \dots & a_{n,m}^l \end{bmatrix}$$

Ecuaciones generales | Repr. matricial (cont.)

Cálculo de los **deltas**.

$$\delta^L = (a^L - y) \odot \sigma'(z^L)$$

$$\delta^l = (\mathbf{W}^{l+1})^T \delta^{l+1} \odot \sigma'(z^l)$$

Actualización de los pesos y sesgos con **descenso del gradiente en mini lotes** (*mini-batch gradient descent*).

$$w_{i,j}^l = w_{i,j}^l - \mu \left(\frac{1}{m} \sum_{k=1}^m \frac{\partial L^{(k)}}{\partial w_{i,j}^l} \right)$$

$$b_i^l = b_i^l - \mu \left(\frac{1}{m} \sum_{k=1}^m \frac{\partial L^{(k)}}{\partial b_i^l} \right)$$

Ejercicio red fija

Para los ejercicios de hoy, en vez de PyTorch, vamos a usar **NumPy**.

Veamos `td6-p17-c-backpropagation.ipynb`.

Ejercicio red genérica | Idea del algoritmo

```
1: procedure TRAINNEURALNETWORK( $X, Y$ )
2:    $layers \leftarrow [8, 16, 20, \dots]$ 
3:   Inicializar  $W, b$  con valores aleatorios pequeños
4:    $learning\_rate \leftarrow \mu$ 
5:   for  $epoch \leftarrow 1$  to  $epochs$  do
6:     for cada mini-batch  $(X_{batch}, Y_{batch})$  de  $X$  y  $Y$  do
7:        $(Z, A) \leftarrow \text{FORWARD\_GENERIC}(X_{batch})$ 
8:        $(\delta_W, \delta_b) \leftarrow \text{BACKWARD\_GENERIC}(X_{batch}, Y_{batch}, A, Z)$ 
9:        $\text{UPDATE\_PARAMETERS\_GENERIC}(\delta_W, \delta_b)$ 
10:    end for
11:  end for
12: end procedure
```

Ejercicio red genérica | Idea del algoritmo (cont.)

```
1: procedure FORWARD_GENERIC( $X$ )
2:    $Z \leftarrow []$  Lista vacía para almacenar las matrices  $Z$ 
3:    $A \leftarrow [X]$  Lista con  $X$  como su primer elemento
4:   for cada capa desde la entrada hasta la penúltima do
5:     Calcula las entradas netas  $Z$  de la capa usando la última activación
      en  $A$ , los pesos  $W$  y el sesgo  $b$ 
6:     Calcula las activaciones  $A$  usando la función LeakyReLU y las en-
      tradas ponderadas  $Z$ 
7:     Almacena las entradas ponderadas y las activaciones en sus respec-
      tivas listas
8:   end for
9:   return Listas de entradas ponderadas  $Z$  y activaciones  $A$ 
10: end procedure
```

Ejercicio red genérica | Idea del algoritmo (cont.)

```
1: procedure BACKWARD_GENERIC(X, Y, A, Z)
2:   Inicializa listas vacías para  $\delta_Z$ ,  $\delta_W$ , y  $\delta_b$ 
3:   Calcula el error de la última capa usando la diferencia entre la última
    activación y Y
4:   for cada capa desde la última hasta la primera do
5:     Calcula el error de la capa usando el error de la siguiente capa y los
    pesos
6:     Calcula el gradiente del costo con respecto a los pesos y sesgos usando
    el error de la capa y las activaciones
7:     Almacena los gradientes en sus respectivas listas
8:   end for
9:   return Listas de gradientes para pesos  $\delta_W$  y sesgos  $\delta_b$ 
10: end procedure
```


Ejercicio red genérica | Código

Volvamos a `td6-p17-c-backpropagation.ipynb`.

Cierre

Hoy vimos y practicamos,

- cómo se traducen las ecuaciones de backpropagation a representación **matricial**;
- cómo se programa esta estrategia de backpropagation con **NumPy** y
- cómo se implementa para un **conjunto de datos específico** de forma general para **N capas y M neuronas**.

Esta semana, la clase de **consultas** será el **miércoles** a las **18 h**.

Ya se encuentra abierto el registro de grupos para el **TP3**.

Pueden darnos *feedback* de la clase [acá](#).